PROYECTO FINAL · DEEP LEARNING Y SISTEMAS INTELIGENTES · UNIVERSIDAD DEL VALLE · 2026

Puntaje de riesgo de fraude transaccional en producción

Modelo de origen: Proyecto 1, *Monitoreo transaccional* (LSTM sobre la secuencia de compras)

> **Pregunta para el comité:** ¿se mantiene este modelo en producción?

Jonathan Zacarías · Luis Gilberto González · Diego López

Servicio: **fraude-api-t2ww.onrender.com**

1 · PROBLEMÁTICA

Decidir en milisegundos qué hacer con cada compra con tarjeta

- **Usuario:** el motor de autorización del banco y los analistas de prevención de fraude.
- **Pregunta del Proyecto 1:** ¿el orden de las compras aporta información que los promedios no capturan, y cuánto vale en quetzales?
- **Alcance:** el servicio da un puntaje y una acción sugerida. No explica el patrón de fraude.

Fraude no detectado
Q4,200

Bloqueo a un cliente legítimo
Q180

Decisión que depende del puntaje:

Aprobar	**Verificar**	**Bloquear**
sin fricción	OTP o biometría	tarjeta y alerta a analista

Fuente: README, sección 1

Del notebook al servicio público en una semana

FECHA	ETAPA	RESPONSABLE	ESTADO
7 nov	Elegir el Proyecto 1 y revisar notebook y contrato	Todos	Hecho
7 nov	Extraer el pipeline a un paquete; serializar escaladores y encoders	Todos	Hecho
7 nov	API (FastAPI), política de decisión y registro de uso	Todos	Hecho
8 nov	Verificar pesos y métricas con `export_artifacts.py`	Jonathan Zacarías	Hecho
8 nov	Registro persistente, integración continua y publicación en Render	Diego López	Hecho
13 nov	Pruebas con persona ajena, capturas y registro de uso	Todos	Hecho
13 nov	Presentación en PDF	Todos	Hecho

Entrega límite: 15 de noviembre, 23:59.

Fuente: README, sección 2

3 · SUPUESTOS

Lo que damos por cierto, y dónde puede fallar

Datos

Sintéticos: 1,200 clientes, 90 días, semilla 42, con mecanismos de fraude inyectados.

Entre 9.1% y 9.6% de las filas son fraude.

Partición temporal: días 1–60 entrenamiento, 61–75 validación, 76–90 prueba.

No se probó con datos reales.

Usuario y costos

Costos de error del Proyecto 1: Q4,200 por fraude no detectado y Q180 por bloqueo injusto.

El sistema que llama a la API puede enviar las últimas 6 compras del cliente y su monto promedio.

Entorno

Inferencia en CPU, una transacción por petición.

Render, plan gratuito: 0.1 CPU y 512 MB.

Tras 15 minutos sin tráfico se duerme y tarda cerca de un minuto en despertar.

Fuente: README, sección 3

4 · MODELO PLANTEADO

Tres modelos del Proyecto 1; en producción servimos B

A · Agregado

MLP sobre 12 variables agregadas. Representa el sistema actual del banco simulado.

Línea base

B · Secuencial en producción

LSTM con embeddings de comercio y canal sobre las últimas **6** transacciones del cliente.

C · Híbrido

Dos ramas: la secuencia de B y las variables de A.

Se puede servir con `MODEL=c` sin cambiar nada más.

No se reentrenó ningún modelo. Se sirven los pesos publicados en `artefactos/`, y `export_artifacts.py` reproduce sus métricas con diferencia 0.

Fuente: README, secciones 4 y 5.2

Qué cambiamos para producción y qué encontramos

Cambios justificados

- **Pipeline serializado** (`pipeline.json`): el notebook lo ajustaba en memoria.
- **Variables calculadas en el servidor:** el contrato pedía 6 variables y el modelo usa 12. El cliente solo envía sus compras.
- **Umbral de revisión = óptimo de validación** (0.01 para B). La política del P1 usaba 0.18, que no coincidía. Bloqueo en 0.65, tomado de esa política.
- **Se quitaron** `detected_pattern` y `confidence`: el modelo no los produce.

Limitaciones que encontramos

- La razón monto/promedio quedó contaminada (primera compra con promedio 0). Afecta a A y a la rama agregada de C.
- En el notebook, el **16.6%** de las ventanas de prueba estaban cortadas. Con historial completo, B cuesta Q31,140 en prueba en vez de Q35,520.
- Puntajes saturados: el **97.8%** de B es < 0.001 o > 0.999. Ordena el riesgo, no es una probabilidad.
- En prueba, B aprueba 90.2%, bloquea 9.8% y verifica solo 0.02%: casi binario.

Fuente: README, sección 4

5 · VALIDACIÓN DE HIPÓTESIS

Hipótesis: el orden de las compras aporta valor

Criterio (del notebook): ΔAUC-PR de C frente a A ≥ +0.05 **y** ahorro ≥ Q15,000 al mes. Aparece en las conclusiones del notebook, no antes de entrenar; no lo presentamos como fijado de antemano.

SISTEMA (PRUEBA, 15 DÍAS)	UMBRAL	AUC-PR	RECALL	PRECISIÓN	FALSOS +	FALSOS −	COSTO
Ingenuo (A a 0.50)	0.50	0.9811	99.4%	71.3%	1,385	20	Q333,300
A · línea base	0.36	0.9811	99.7%	70.2%	1,459	11	Q308,820
B · secuencial	**0.01**	**0.9986**	**99.9%**	**97.1%**	**104**	**4**	**Q35,520**
C · híbrido	0.09	0.9986	99.9%	97.4%	91	4	Q33,180

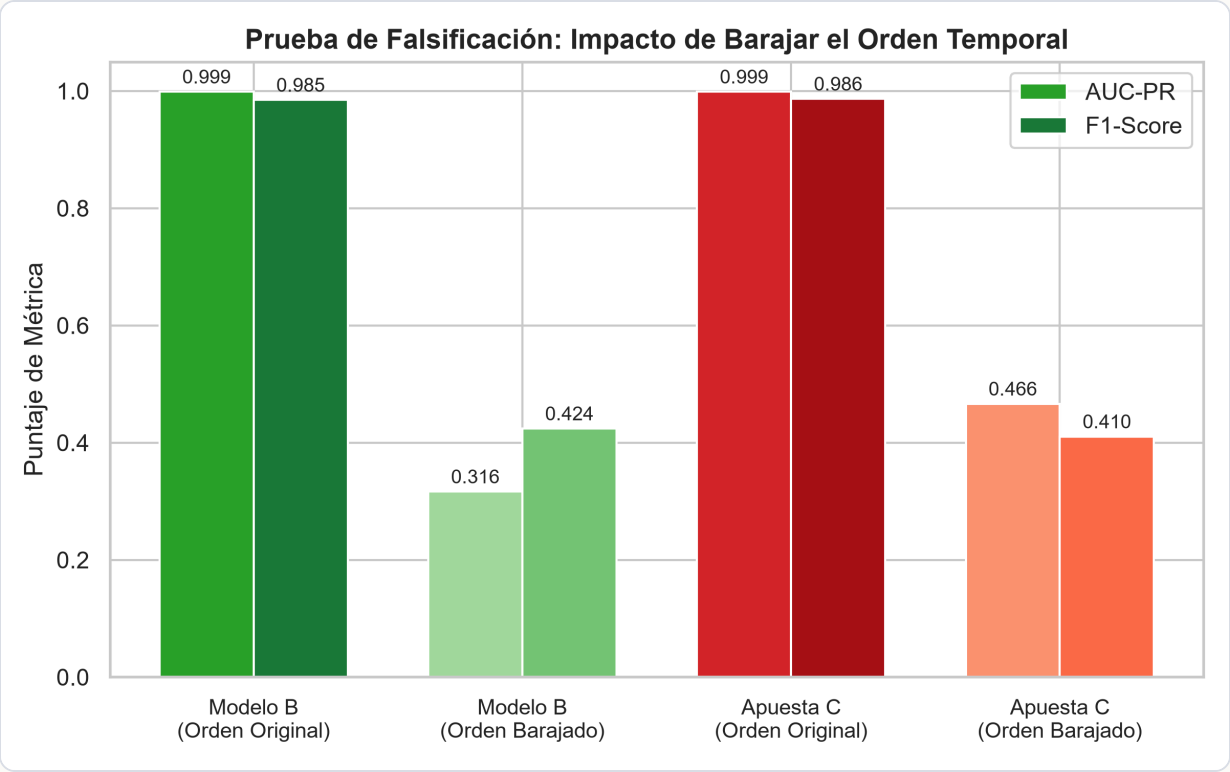
No se cumple ΔAUC-PR de C frente a A: **+0.018** (pedía +0.05).

Se cumple Ahorro de C frente a A: **Q551,280 al mes**.

Resultado: como el criterio exige ambas, la hipótesis **no se cumple tal como fue planteada**. El valor del orden aparece en el costo, no en el AUC-PR.

Fuente: artefactos/metricas_completas.json, reproducido por export_artifacts.py · README 5.1

Si se baraja el orden, el modelo se cae

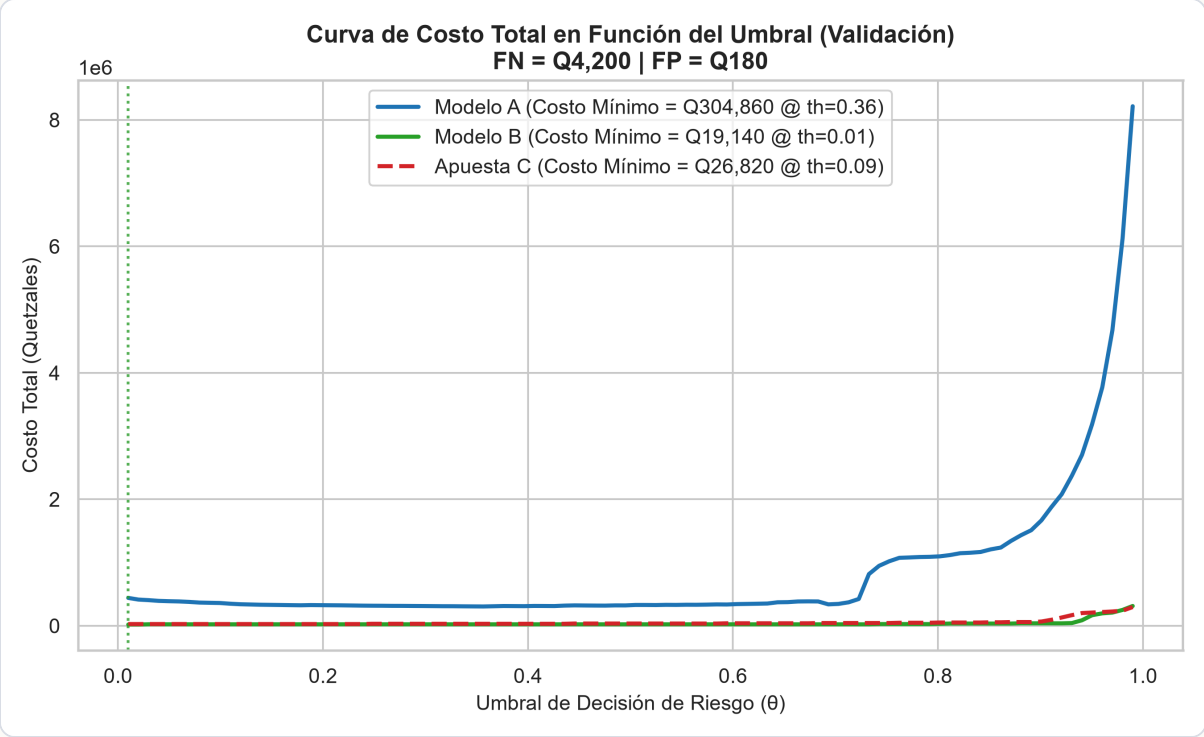


- Al barajar la secuencia, el AUC-PR de **B cae de 0.999 a 0.316** y el de C a 0.466.
- Frente a A, B baja los falsos positivos de **1,459 a 104** y los falsos negativos de 11 a 4.
- Ahorro mensual frente a A: **B Q546,600** · C Q551,280.

> **Cuidado al leerlo:** los datos sintéticos son fáciles de separar; hasta A llega a 0.98. Estos números no anticipan el desempeño con datos reales.

Fuente: artefactos/prueba_permutacion.png · README 5.1

Por qué servimos B y no C



- **Un modelo se elige con validación:** B cuesta Q19,140 y C Q26,820.
- En prueba empatan en AUC-PR (diferencia 0.00001). C cuesta Q2,340 menos, que son pocos casos y no se pueden usar para elegir.
- B es más simple: no usa la rama agregada contaminada.

> **Riesgo aceptado:** el umbral de B (0.01) está en el borde de la búsqueda; pequeños cambios en los puntajes lo afectan. Por eso vigilamos la proporción de bloqueos en `/stats`.

Fuente: artefactos/curva_costo_economico.png · README 5.1

Datos, entrenamiento, servicio y monitoreo



Por qué Supabase: el plan gratuito de Render borra los archivos locales al dormir o redesplegar. El registro se guarda en segundo plano y no guarda `client_id` ni montos.

Herramientas: PyTorch (CPU), FastAPI y Uvicorn, Docker, Render (0.1 CPU, 512 MB), GitHub Actions, Supabase.

Fuente: README, sección 6

¿El servicio hace lo mismo que el notebook?

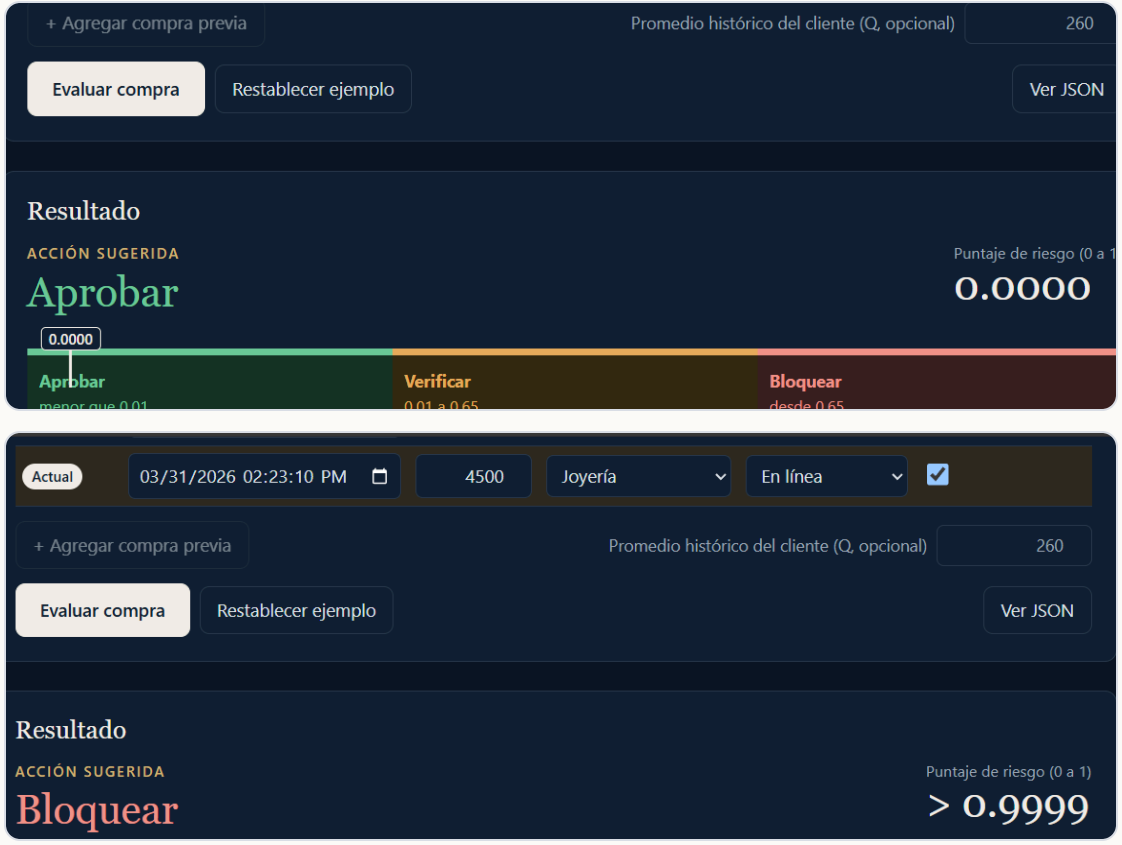
HIPÓTESIS	CRITERIO	RESULTADO
La API puntúa igual que el notebook	error ≤ 1e-5	Cumplido 5.96e-08 en 150 transacciones por modelo
La API construye las mismas variables	≤ 1e-4 y 0 ids distintos	Cumplido error 0 en 400 transacciones
Los pesos reproducen lo reportado	AUC-PR, diferencia ≤ 0.02	Cumplido diferencia 0 con `metricas_completas.json`
Latencia en línea (contrato original)	p95 < 25 ms	No cumplido 371 ms de extremo a extremo desde Guatemala; el modelo solo, 1.4 ms

Origen de los criterios: latencia del contrato del P1; tolerancias en el código de las pruebas desde el 7 nov; el 0.02 de AUC-PR se fijó junto con la medición.

Monitoreo propuesto (últimas 500 peticiones): alerta si revisión+bloqueo sale de 5%–15%, verificación > 2%, latencia del modelo p95 > 25 ms o errores 5xx > 1%. **Sin etiquetas en tiempo real no se puede medir AUC-PR en línea:** harían falta contracargos confirmados.

Fuente: README, secciones 5.2 y 6

Está publicado, probado y en uso



Capturas desde la laptop de una persona ajena: "Cliente habitual" → Aprobar (0.0000) · "Sondeo y vaciado" → Bloquear (> 0.9999).

fraude-api-t2ww.onrender.com

Uso real (8 al 13 nov, sin las 178 filas de pruebas automáticas)

69	1.24	5.34
peticiones	ms, p50 del modelo	ms, p95 del modelo

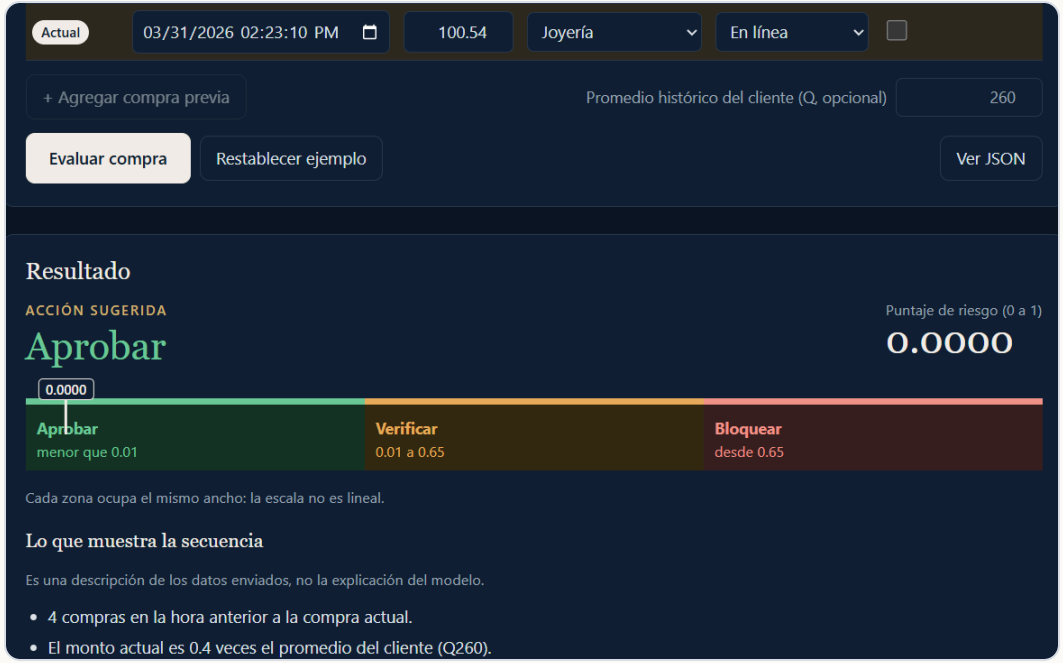
39.1%	**18.8%**	**42.0%**
Aprobar (27)	Verificar (13)	Bloquear (29)

Pruebas: 4 pruebas automáticas pasan en cada cambio (equivalencia de variables y de puntaje, capa HTTP y pesos reales). Render despliega solo si pasan.

El 42% de bloqueos viene de los ejemplos de la página, elegidos para mostrar las tres decisiones; no indica deriva.

Fuente: evidencias/ y evidencias/requests.csv · README, sección 7

Mateo Sandoval lo usó sin explicación previa



13 nov, su laptop (Windows 11, Chrome). Bajó la compra de Q4,500 a Q100.54 y quitó "internacional": el sistema la aprobó.

Lo que entendió

- Verde = la compra se parece a lo que el cliente gasta normalmente.
- Por qué se bloqueó la segunda: varias compras en una hora, monto alto y en el extranjero.

Lo que le confundió

- El nombre "Sondeo y vaciado".
- "Acierto: era fraude…": dudó si la respuesta estaba programada o se calculaba en el momento.
- "No es la explicación del modelo", justo encima de una lista de motivos.

> **Hallazgo sobre el modelo:** con los mismos 4 cargos pequeños en una hora, una compra final pequeña y nacional se aprueba. El sondeo por sí solo no basta para que B la marque.

Fuente: README, sección 7 · evidencias/05 a 07

DECISIÓN PARA EL COMITÉ

Nuestra recomendación: mantener B en producción como piloto

A favor

- Reproduce exactamente al notebook (error 5.96e-08) y los pesos publicados (diferencia 0).
- Frente al sistema actual (A), Q546,600 menos de costo al mes en prueba y 1,355 bloqueos injustos menos en 15 días.
- Publicado, usado por una persona ajena y monitoreado.

Lo que no está resuelto

- Datos sintéticos: falta validar con datos reales.
- No cumple el criterio de AUC-PR ni la latencia de 25 ms de extremo a extremo.
- Umbral en el borde (0.01) y puntajes saturados.

Condiciones para salir del piloto: conectar contracargos confirmados para medir recall y costo reales, recalibrar el umbral con esos datos y servir cerca del motor de autorización para bajar la latencia de red.

Gracias. ¿Preguntas? fraude-api-t2ww.onrender.com · github.com/Zacatac23/ProyectoFinalDeepLearning